Assignment 2: LiveOak-2 to SaM Compiler

C S 395T SIMPL, Summer 2022
Unique ID: 86484
TAs: Tyler Collins, Peter Gao, Aaron Wollman

Assigned: Monday, 20 June 2022 00:01 CT
Due: Sunday, 17 July 2021 23:59 CT
Last possible hand in: Tuesday, 19 July 2021 23:59 CT

1 Introduction

This programming assignment involves implementing a compiler that takes a simple Java-like language called LiveOak (described below) as input and generates code for a stack machine named SaM (described below). The compiler will use a recursive-descent parser. The assignment is intended to help you better understand recursive-descent parsing and get a feel of what it means to implement a compiler.

The following are links to the relevant materials in the lecture videos: LiveOak; stack machines and SaM; recursive-descent parsing; and code generation.

2 Logistics

You may use up to two late (slip) days for this assignment. Start early enough to complete your work before the due date. Assume things will not go according to plan, and so you must allow extra time for heavily loaded systems, dropped internet connections, corrupted files, traffic delays, minor health problems, etc. Do not request an extension—it will not be granted.

Every project in C S 395T is an individual project. Before you begin, please take the time to review the course policy on academic integrity at: `https://www.cs.utexas.edu/academics/conduct`.

All hand-ins are electronic. You can do this assignment on any machine you choose. The testing for your submission is automated on Gradescope.

3 Details of the assignment

Your task is to create a handwritten recursive-descent parser and SaM code generator for levels 0, 1, and 2 of the LiveOak language. Section 7 contains the formal grammar of the input language. Sections 7.1 through 7.3 are the grammars for each level. Disregard Section 7.4 for this assignment. It is there only for reference right now. You will need it for Programming Assignment 3.

You will use the supplied `SaMTokenizer` class for lexical analysis. The compiler should take a file containing a LiveOak program as input and produce an output file containing a SaM program that is a correct translation of the input program.

3.1 Download and preparation

You will need the following materials:

- The SaM library (v 2.6.3) containing the lexer, which you already have from Programming Assignment 1. Compile your code with this `.jar`. If you will be using an IDE, you will need to add this `.jar` file as an external library in your project.
- A collection of public testcases. The collection of testcases is still being expanded, and we will keep you updated on Piazza.

Optionally, you can also download the following materials:

- A starter template for your compiler, from here. The template is provided to give you some initial idea on your implementation, but you are free to modify it or develop in your own style altogether. It shows various ways the `SaMTokenizer` class can be used, and also contains methods to help you get started on the project. It contains lots of `TODO`s that you would need to implement.
- The HTML documentation of the SaM API, from here. It provides information on the usage of specific library functions.
- The design document containing the complete workings of the SaM interpreter, from here. It describes how to use the SaM UI, various options for input/output, a description of the SaM ISA, and so on.

You can use the documentation and the design document even though they are down-version.

3.2 Additional information regarding implementation

Here are some additional assertions regarding the grammar and the language.

1. There will always be a `main` method in the input program. The program starts executing from the `main` method. The `main` method does not take any arguments.
2. There will always be a `return` statement at the end of a method in the input program.
3. Comments in the input program are automatically handled by the `SamTokenizer` class. The tokenizer discards characters between `//` and the end of the line (including `//`) and you do not need to worry about it.
4. There is no overloading of methods.
5. The definition of a method can occur either before or after calls to it. You will need a symbol table for each method. One approach would be to have a separate class for the symbol table (using hash tables or any approach). A symbol table object would be created inside your `getMethod()` method, and be initialized by the `getDeclarations()` method call. Once initialized it would be passed to all (almost) other method invocations inside `getMethod()` to make sure each rule has the appropriate information. Each method would have its own symbol table.
6. A `break` statement must be lexically nested within one or more loops, and when it is executed, it terminates the execution of the innermost loop in which it is nested. You are required to handle illegal break statements.

7. If a program does not satisfy the grammar above or does not satisfy the textual description of the language, your compiler should print a short, informative error message and/or exit with a non-zero exit status. If you have any question or confusion, post on Piazza.

Make sure that your compiler is in the Java class `assignment2.LiveOak2Compiler`. Your compiler should take two command-line arguments. The first argument is an input file containing a LiveOak program. The second argument is an output file that will contain your generated SaM code.

It is recommended (but by no means required) that you manage complexity by working up through the levels of the source language. If you go down this path, you will likely find that you will spend a lot of time in LiveOak-0, relatively little additional time in LiveOak-1, and a moderate amount of time in LiveOak-2.

Don't code indiscriminately. Put effort into design up-front. That investment will pay off handsomely during coding and testing.

4 Hand-in

Submit (to Gradescope) the following:

1. `compiler.jar`: A runnable `.jar` file of your project. Please refer to online resources to learn how to create a runnable `.jar` file for your Java project. Make sure that your `.jar` file can be executed using the command line mentioned in the evaluation section below.

2. `source.zip`: A `.zip` file containing all your source files, including any libraries you may have used. Ideally, we should be able to re-create the `.jar` file from the source, if needed.

It is possible that we may fine-tune the hand-in process. Should this happen, the changes will be announced on Piazza.

5 Evaluation

The following sequence of commands will be used to evaluate your submission on both public and private test cases.

1. Compiling a LiveOak-2 program:
   ```
   java -jar compiler.jar test1.lo output.sam
   ```
   The above command should read the source LiveOak-2 program in the file `test1.lo`, generate the SaM program, and write it to the file `output.sam`.

2. Running a SaM program:
   ```
   java -cp SaM-2.6.3.jar edu.utexas.cs.sam.ui.SamText output.sam
   ```
   The above command reads the SaM program in the file `output.sam` and executes the SaM interpreter. The output of the program, i.e., the exit status of the program, will be displayed on the terminal. From the SaM interpreter's perspective, the exit status is the element on the top of the stack when the interpreter executes the `STOP` instruction. From the LiveOak program's perspective, the exit status is the return value of the `main()` method. The exit status is used to evaluate the correctness of your compiler. An example output of this command is shown below:

```
Program assembled.
Program loaded. Executing.
==========================
Exit Status: 30
```

If there is an error in the SaM program, an error message will be displayed instead of the exit status.

The public test cases provided is not exhaustive and only covers simple scenarios. You are highly recommended to test your compiler by creating more test cases and using the above commands.

Before submitting your `.jar` file, please execute the above commands and make sure the exit status matches the expected output on all test cases. This check will ease the grading process.

6 Grading

This assignment is auto-graded using Gradescope and counts for 30% of your course grade.

The names of test cases indicates which level of the language they are testing. This is for your information only, in case you want to build your compiler in stages and test along the way; grading will be done only on the LiveOak-2 test cases. Both public and private test cases will be used.

7 The LiveOak Grammar

The following comments apply in general.

- Tokens are shown thus: `token`. Nonterminals are shown thus: ⟨*NonTerminal*⟩. Repetition and closure operations are always parenthesized, with the parentheses and the operators shown in blue.[1] Changes from one language level to the next are shown thus: **added** and **deleted** productions.
- In production P22 (the ternary choice operator), evaluation proceeds as in C or Java: the condition is first evaluated, and only one choice is evaluated depending on the value of the condition.
- In productions P23 and P28, the interpretation of the binary operation depends on the types of the input operands.
  - For integers, `+-*/%` are sum, difference, product, quotient, and remainder; and `<>=` are the comparisons less-than, greater-than, and equal-to.
  - For Booleans, `&` is AND and `|` is inclusive-OR. Evaluation is "short-circuit", as in Java and C.
  - For strings, `+` is concatenation; `*` is repetition (with the second argument being an integer); and `<>=` compare the lexicographic ordering of two strings.
  - No other combination is legal.
- In productions P24 and P29, the interpretation of the unary operation depends on the type of the input operand.

[1] Thus, the left parenthesis token that is used in expressions is shown as `(`, while the left parenthesis that is used as part of the repetition structure for variable declaration is shown as (. Likewise for `*` (the multiplication operator token) and * (the closure operator).

  - For integers, ~ is negation.
  - For Booleans, ! is NOT.
  - For strings, ~ is string reversal.
  - No other combination is legal.

- Productions P42–P44 use standard `grep` syntax. ␣ is the whitespace character.
- In production P43, the characters making up the string can be any ASCII characters, including the standard escape sequences used in C.

7.1 LiveOak-0: Expressions, Assignment Statements, Sequential Control Flow

Id	Production	Comments
P1		
P2		
P3	⟨*Program*⟩ → ⟨*Body*⟩	Start here.
P4		
P5		
P6	⟨*Body*⟩ → (⟨*VarDecl*⟩)* ⟨*Block*⟩	
P7		
P8		
P9	⟨*VarDecl*⟩ → ⟨*Type*⟩ ⟨*Identifier*⟩ (`,` ⟨*Identifier*⟩)* `;`	Only declaration; no initialization.
P10	⟨*Block*⟩ → `{` (⟨*Stmt*⟩)+ `}`	Extraneous braces to anticipate later levels.
P11		
P12		
P13		
P14		
P15	⟨*Stmt*⟩ → ⟨*Var*⟩ `=` ⟨*Expr*⟩ `;`	Basic assignment statement.
P16	⟨*Stmt*⟩ → `;`	Null statement.
P17		
P18		
P19		
P20		
P21		
P22	⟨*Expr*⟩ → `(` ⟨*Expr*⟩ `?` ⟨*Expr*⟩ `:` ⟨*Expr*⟩ `)`	Ternary choice operation.
P23	⟨*Expr*⟩ → `(` ⟨*Expr*⟩ ⟨*Binop*⟩ ⟨*Expr*⟩ `)`	Binary operations.
P24	⟨*Expr*⟩ → `(` ⟨*Unop*⟩ ⟨*Expr*⟩ `)`	Unary operations.
P25	⟨*Expr*⟩ → `(` ⟨*Expr*⟩ `)`	Extra parentheses.
P26	⟨*Expr*⟩ → ⟨*Var*⟩	A named variable.
P27	⟨*Expr*⟩ → ⟨*Literal*⟩	A literal value.
P28	⟨*Binop*⟩ → `[+-*/%&|<>=]`	Valid binary operators.
P29	⟨*Unop*⟩ → `[~!]`	Valid unary operators.
P30		
P31		
P32	⟨*Type*⟩ → `int`	
P33	⟨*Type*⟩ → `bool`	
P34	⟨*Type*⟩ → `String`	
P35	⟨*Literal*⟩ → ⟨*Num*⟩	Integer literals. Non-negative only.
P36	⟨*Literal*⟩ → `true`	Boolean literal `true`.
P37	⟨*Literal*⟩ → `false`	Boolean literal `false`.
P38	⟨*Literal*⟩ → ⟨*String*⟩	A string literal.
P39		
P40		
P41	⟨*Var*⟩ → ⟨*Identifier*⟩	
P42	⟨*Num*⟩ → `([0-9])+`	Regular expression for integer literals.
P43	⟨*String*⟩ → `"` (*ASCII character*)* `"`	Regular expression for string literals.
P44	⟨*Identifier*⟩ → `[a-zA-Z]([a-zA-Z0-9'_'])*`	Regular expression for identifiers.

7.2 LiveOak-1: Imperative Programming with Structured Control Flow

Id	Production	Comments
P1		
P2		
P3	⟨*Program*⟩ → ⟨*Body*⟩	
P4		
P5		
P6	⟨*Body*⟩ → (⟨*VarDecl*⟩)* ⟨*Block*⟩	
P7		
P8		
P9	⟨*VarDecl*⟩ → ⟨*Type*⟩ ⟨*Identifier*⟩ (`,` ⟨*Identifier*⟩)* `;`	
P10	⟨*Block*⟩ → `{` (⟨*Stmt*⟩)+ `}`	
P11		
P12	⟨*Stmt*⟩ → `if` `(` ⟨*Expr*⟩ `)` ⟨*Block*⟩ `else` ⟨*Block*⟩	Conditional statement.
P13	⟨*Stmt*⟩ → `while` `(` ⟨*Expr*⟩ `)` ⟨*Block*⟩	Loop statement.
P14	⟨*Stmt*⟩ → `break` `;`	Break out of enclosing loop.
P15	⟨*Stmt*⟩ → ⟨*Var*⟩ `=` ⟨*Expr*⟩ `;`	
P16	⟨*Stmt*⟩ → `;`	
P17		
P18		
P19		
P20		
P21		
P22	⟨*Expr*⟩ → `(` ⟨*Expr*⟩ `?` ⟨*Expr*⟩ `:` ⟨*Expr*⟩ `)`	
P23	⟨*Expr*⟩ → `(` ⟨*Expr*⟩ ⟨*Binop*⟩ ⟨*Expr*⟩ `)`	
P24	⟨*Expr*⟩ → `(` ⟨*Unop*⟩ ⟨*Expr*⟩ `)`	
P25	⟨*Expr*⟩ → `(` ⟨*Expr*⟩ `)`	
P26	⟨*Expr*⟩ → ⟨*Var*⟩	
P27	⟨*Expr*⟩ → ⟨*Literal*⟩	
P28	⟨*Binop*⟩ → `[+-*/%&|<>=]`	
P29	⟨*Unop*⟩ → `[~!]`	
P30		
P31		
P32	⟨*Type*⟩ → `int`	
P33	⟨*Type*⟩ → `bool`	
P34	⟨*Type*⟩ → `String`	
P35	⟨*Literal*⟩ → ⟨*Num*⟩	
P36	⟨*Literal*⟩ → `true`	
P37	⟨*Literal*⟩ → `false`	
P38	⟨*Literal*⟩ → ⟨*String*⟩	
P39		
P40		
P41	⟨*Var*⟩ → ⟨*Identifier*⟩	
P42	⟨*Num*⟩ → `([0-9])`+	
P43	⟨*String*⟩ → `"` `(['␣'-'~'])*` `"`	
P44	⟨*Identifier*⟩ → `[a-zA-Z]([a-zA-Z0-9'_'])*`	

7.3 LiveOak-2: Procedural Programming

Id	Production	Comments
P1		
P2	⟨*Program*⟩ → (⟨*MethodDecl*⟩)*	Start here.
P3		Not here.
P4		
P5	⟨*MethodDecl*⟩ → ⟨*Type*⟩ ⟨*MethodName*⟩ `(` (⟨*Formals*⟩)? `)` `{` ⟨*Body*⟩ `}`	Method declaration.
P6	⟨*Body*⟩ → (⟨*VarDecl*⟩)* ⟨*Block*⟩	
P7	⟨*Formals*⟩ → ⟨*Type*⟩ ⟨*Identifier*⟩ (`,` ⟨*Type*⟩ ⟨*Identifier*⟩)*	Formal parameters in a method declaration.
P8	⟨*Actuals*⟩ → ⟨*Expr*⟩ (`,` ⟨*Expr*⟩)*	Actual parameters in a method invocation.
P9	⟨*VarDecl*⟩ → ⟨*Type*⟩ ⟨*Identifier*⟩ (`,` ⟨*Identifier*⟩)* `;`	
P10	⟨*Block*⟩ → `{` (⟨*Stmt*⟩)+ `}`	
P11	⟨*Stmt*⟩ → `return` ⟨*Expr*⟩ `;`	Return a result value from a method.
P12	⟨*Stmt*⟩ → `if` `(` ⟨*Expr*⟩ `)` ⟨*Block*⟩ `else` ⟨*Block*⟩	
P13	⟨*Stmt*⟩ → `while` `(` ⟨*Expr*⟩ `)` ⟨*Block*⟩	
P14	⟨*Stmt*⟩ → `break` `;`	
P15	⟨*Stmt*⟩ → ⟨*Var*⟩ `=` ⟨*Expr*⟩ `;`	
P16	⟨*Stmt*⟩ → `;`	
P17		
P18		
P19		
P20		
P21	⟨*Expr*⟩ → ⟨*MethodName*⟩ `(` (⟨*Actuals*⟩)? `)`	Invoke a method.
P22	⟨*Expr*⟩ → `(` ⟨*Expr*⟩ `?` ⟨*Expr*⟩ `:` ⟨*Expr*⟩ `)`	
P23	⟨*Expr*⟩ → `(` ⟨*Expr*⟩ ⟨*Binop*⟩ ⟨*Expr*⟩ `)`	
P24	⟨*Expr*⟩ → `(` ⟨*Unop*⟩ ⟨*Expr*⟩ `)`	
P25	⟨*Expr*⟩ → `(` ⟨*Expr*⟩ `)`	
P26	⟨*Expr*⟩ → ⟨*Var*⟩	
P27	⟨*Expr*⟩ → ⟨*Literal*⟩	
P28	⟨*Binop*⟩ → `[+-*/%&|<>=]`	
P29	⟨*Unop*⟩ → `[~!]`	
P30		
P31		
P32	⟨*Type*⟩ → `int`	
P33	⟨*Type*⟩ → `bool`	
P34	⟨*Type*⟩ → `String`	
P35	⟨*Literal*⟩ → ⟨*Num*⟩	
P36	⟨*Literal*⟩ → `true`	
P37	⟨*Literal*⟩ → `false`	
P38	⟨*Literal*⟩ → ⟨*String*⟩	
P39		
P40	⟨*MethodName*⟩ → ⟨*Identifier*⟩	
P41	⟨*Var*⟩ → ⟨*Identifier*⟩	
P42	⟨*Num*⟩ → `([0-9])+`	
P43	⟨*String*⟩ → `"` `([' '-'~'])*` `"`	
P44	⟨*Identifier*⟩ → `[a-zA-Z]([a-zA-Z0-9'_'])*`	

7.4 LiveOak-3: Objects and Classes, No Inheritance

Id	Production	Comments
P1	⟨*Program*⟩ → (⟨*ClassDecl*⟩)*	Start here.
P2		Not here.
P3		
P4	⟨*ClassDecl*⟩ → `class` ⟨*ClassName*⟩ `(` (⟨*VarDecl*⟩)* `)` `{` (⟨*MethodDecl*⟩)* `}`	Class declaration.
P5	⟨*MethodDecl*⟩ → ⟨*Type*⟩ ⟨*MethodName*⟩ `(` (⟨*Formals*⟩)? `)` `{` ⟨*Body*⟩ `}`	
P6	⟨*Body*⟩ → (⟨*VarDecl*⟩)* ⟨*Block*⟩	
P7	⟨*Formals*⟩ → ⟨*Type*⟩ ⟨*Identifier*⟩ (`,` ⟨*Type*⟩ ⟨*Identifier*⟩)*	
P8	⟨*Actuals*⟩ → ⟨*Expr*⟩ (`,` ⟨*Expr*⟩)*	
P9	⟨*VarDecl*⟩ → ⟨*Type*⟩ ⟨*Identifier*⟩ (`,` ⟨*Identifier*⟩)* `;`	
P10	⟨*Block*⟩ → `{` (⟨*Stmt*⟩)+ `}`	
P11	⟨*Stmt*⟩ → `return` ⟨*Expr*⟩ `;`	
P12	⟨*Stmt*⟩ → `if` `(` ⟨*Expr*⟩ `)` ⟨*Block*⟩ `else` ⟨*Block*⟩	
P13	⟨*Stmt*⟩ → `while` `(` ⟨*Expr*⟩ `)` ⟨*Block*⟩	
P14	⟨*Stmt*⟩ → `break` `;`	
P15	⟨*Stmt*⟩ → ⟨*Var*⟩ `=` ⟨*Expr*⟩ `;`	
P16	⟨*Stmt*⟩ → `;`	
P17	⟨*Expr*⟩ → `this`	Reference to current object.
P18	⟨*Expr*⟩ → `null`	A null object.
P19	⟨*Expr*⟩ → `new` ⟨*ClassName*⟩ `(` (⟨*Actuals*⟩)? `)`	Create a new object.
P20	⟨*Expr*⟩ → ⟨*Var*⟩`.` ⟨*MethodName*⟩ `(` (⟨*Actuals*⟩)? `)`	Invoke an instance method.
P21		No static methods.
P22	⟨*Expr*⟩ → `(` ⟨*Expr*⟩ `?` ⟨*Expr*⟩ `:` ⟨*Expr*⟩ `)`	
P23	⟨*Expr*⟩ → `(` ⟨*Expr*⟩ ⟨*Binop*⟩ ⟨*Expr*⟩ `)`	
P24	⟨*Expr*⟩ → `(` ⟨*Unop*⟩ ⟨*Expr*⟩ `)`	
P25	⟨*Expr*⟩ → `(` ⟨*Expr*⟩ `)`	
P26	⟨*Expr*⟩ → ⟨*Var*⟩	
P27	⟨*Expr*⟩ → ⟨*Literal*⟩	
P28	⟨*Binop*⟩ → `[+-*/%&|<>=]`	
P29	⟨*Unop*⟩ → `[~!]`	
P30	⟨*Type*⟩ → `void`	Return type of constructor.
P31	⟨*Type*⟩ → ⟨*ClassName*⟩	User-defined type.
P32	⟨*Type*⟩ → `int`	
P33	⟨*Type*⟩ → `bool`	
P34	⟨*Type*⟩ → `String`	
P35	⟨*Literal*⟩ → ⟨*Num*⟩	
P36	⟨*Literal*⟩ → `true`	
P37	⟨*Literal*⟩ → `false`	
P38	⟨*Literal*⟩ → ⟨*String*⟩	
P39	⟨*ClassName*⟩ → ⟨*Identifier*⟩	
P40	⟨*MethodName*⟩ → ⟨*Identifier*⟩	
P41	⟨*Var*⟩ → ⟨*Identifier*⟩	
P42	⟨*Num*⟩ → `([0-9])+`	
P43	⟨*String*⟩ → `"` `([' '-'~'])*` `"`	
P44	⟨*Identifier*⟩ → `[a-zA-Z]([a-zA-Z0-9'_'])*`	